

Guida Completa per Crocette - SWE

Indice

1. [SOLID Principles](#)
 2. [Coesione e Accoppiamento](#)
 3. [Design Pattern](#)
 4. [Metriche di Progetto](#)
 5. [Milestone e Baseline](#)
 6. [Diagrammi UML](#)
 7. [Concetti Teorici Fondamentali](#)
 8. [Processi di Ciclo di Vita](#)
 9. [Test e Verifica](#)
 10. [Metodi Agili](#)
 11. [Architetture Software](#)
-

1. SOLID Principles

1.1 Single Responsibility Principle (SRP)

DEFINIZIONE: Una classe dovrebbe avere una, e una sola, ragione per cambiare.

Caratteristiche:

- Noto anche come **coesione**
- Un modulo deve avere correlazione funzionale degli elementi
- Le responsabilità sono accoppiate (modifiche a una possono compromettere altre)
- Analizzare tutto il contesto, non la singola classe

CROCETTA IMPORTANTE: Il Single Responsibility Principle è anche noto con il nome di **coesione** ✓

Esempi:

- SBAGLIATO: Classe che ha draw() per GUI e area() per calcoli
- GIUSTO: Separare logica grafica da logica di calcolo

1.2 Open-Closed Principle (OCP)

DEFINIZIONE: Le entità software devono essere aperte all'estensione, ma chiuse alle modifiche.

Caratteristiche:

- Estendere comportamento aggiungendo nuovo codice, non modificando vecchio
- Astrazione è la chiave
- Variabili private, evitare globali, attenzione a RTTI

CROCETTA IMPORTANTE: È possibile che un gruppo di classi sia Open-Close anche se si utilizzano variabili globali, nei modi e termini permessi dal linguaggio ✓

1.3 Liskov Substitution Principle (LSP)

DEFINIZIONE: Le classi derivate devono essere sostituibili con le loro classi base.

Caratteristiche:

- Base della programmazione ad oggetti
- Precondizioni nelle derivate: uguali o più deboli
- Postcondizioni nelle derivate: uguali o più forti
- Collegato a **Design by Contract**

CROCETTA IMPORTANTE: Usando il Liskov Substitution Principle è associabile al concetto di **Design by Contract** ✓

1.4 Interface Segregation Principle (ISP)

DEFINIZIONE: Creare interfacce a grana fine specifiche per il client.

Caratteristiche:

- Client non deve implementare interfacce che non usa
- Riduce accoppiamento
- Interfacce specializzate e coese

1.5 Dependency Inversion Principle (DIP)

DEFINIZIONE: Si dovrebbe dipendere dalle astrazioni, non dalle concretizzazioni.

Caratteristiche:

- Evitare dipendenze dirette tra alto e basso livello
- Entrambi dipendono da astrazioni
- Policy decisionali nei moduli alto livello

2. Coesione e Accoppiamento

2.1 Coesione

DEFINIZIONE: Proprietà dei singoli componenti che misura quanto le parti concorrano allo stesso obiettivo.

Tipi di Coesione (dal migliore al peggiore):

1. **Funzionale:** parti concorrono allo stesso compito nel tempo
2. **Temporale:** azioni "vicine" per ordine di esecuzione (pipeline)
3. **Informativa:** parti agiscono sulle stesse unità dati (get/set)

Caratteristiche:

- Va **massimizzata**
- Produce maggiore incapsulazione (information hiding)
- Facilita copertura e verifica
- Limita la spinta a decomporre sempre di più

CROCETTA IMPORTANTE: La coesione è una proprietà dei **singoli componenti** che va massimizzata

2.2 Accoppiamento

DEFINIZIONE: Intensità di impatto che i cambiamenti interni a una componente possano avere sull'altra.

Caratteristiche:

- Va **minimizzato**
- Misura dipendenza tra componenti
- Maggiore accoppiamento = maggiore probabilità di modifiche

Tipi:

- **Tightly-coupled:** maggiore probabilità di cambiamenti
- **Loosely-coupled:** minore probabilità di modifiche

CROCETTA IMPORTANTE: L'accoppiamento è caratteristica **negativa** da ridurre, misura l'intensità di impatto dei cambiamenti ✓

2.3 Metriche di Accoppiamento

Fan-in e Fan-out

- **SFIN (Structural Fan-In):** numero di moduli che chiamano questo modulo
- **SFOUT (Structural Fan-Out):** numero di moduli chiamati da questo modulo

Instabilità

FORMULA: $\text{Instabilità} = \text{SFOUT} / (\text{SFIN} + \text{SFOUT})$

- **Valore basso:** forte coesione, scarso accoppiamento (buono)
- **Valore alto:** accoppiamento troppo forte (cattivo)

CROCETTA IMPORTANTE: Se un componente è massimamente utilizzato (alto Fan-in) e minimamente accoppiato (basso Fan-out), allora è **coeso**

3. Design Pattern

3.1 Observer Pattern

Scopo: Definisce dipendenze tra oggetti affinché quando uno cambia stato, tutti i dipendenti siano notificati.

Struttura:

- **Subject:** attach/detach/notify, mantiene lista Observer
- **Observer:** interfaccia con update()
- **ConcreteSubject:** mantiene stato, implementa Subject
- **ConcreteObserver:** mantiene riferimento Subject, implementa update()

CROCETTA IMPORTANTE: Il Subject notifica gli Observer del cambio di stato

3.2 Strategy Pattern

Scopo: Definisce famiglia di algoritmi, li rende intercambiabili.

Struttura:

- **Strategy:** interfaccia comune algoritmi
- **ConcreteStrategy:** implementazioni specifiche
- **Context:** mantiene riferimento Strategy, delega esecuzione

3.3 Template Method Pattern

Scopo: Definisce scheletro algoritmo, sottoclassi personalizzano passi specifici.

Caratteristiche:

- Usa **ereditarietà**, NON composizione
- Ordine operazioni fisso nella superclasse
- Sottoclassi implementano metodi astratti

CROCETTA IMPORTANTE: Il Template Method si richiede di non utilizzare ereditarietà, ma esclusivamente composizione - **FALSO**

3.4 Proxy Pattern

Scopo: Fornisce surrogato per controllare accesso ad altro oggetto.

Tipi:

- **Remote Proxy:** rappresentante locale oggetto remoto
- **Virtual Proxy:** crea oggetti pesanti on-demand
- **Protection Proxy:** controlla accesso
- **Smart Reference:** azioni avanzate

3.5 Singleton Pattern

Scopo: Assicura classe abbia una sola istanza.

CROCETTA IMPORTANTE: Validità sintattica diagrammi Singleton - controllare costruttore privato, getInstance() statico

3.6 Factory Method vs Abstract Factory

Factory Method: Crea oggetti senza specificare classe concreta **Abstract Factory:** Crea famiglie di oggetti correlati

3.7 Altri Pattern Importanti

- **Adapter:** Converte interfacce incompatibili
 - **Decorator:** Aggiunge responsabilità dinamicamente
 - **Command:** Incapsula richiesta come oggetto
-

4. Metriche di Progetto

4.1 Efficacia vs Efficienza

Efficacia

DEFINIZIONE: Grado di conformità del prodotto rispetto alle norme vigenti e agli obiettivi prefissati.

Caratteristiche:

- Misura **qualità** del risultato
- "Fare le cose giuste"
- Richiede sufficienti risorse
- Garantita dal Piano di Qualifica

Efficienza

DEFINIZIONE: Capacità di raggiungere obiettivi perseguendo strategia di risparmio risorse.

Caratteristiche:

- **Funzione inversa** della quantità di risorse impiegate
- "Fare le cose nel modo giusto"
- Indicatore quantitativo consumo risorse

CROCETTA IMPORTANTE:

- Efficacia misura grado di conformità con attese
- Efficienza è funzione inversa delle risorse impiegate ✓

4.2 Ordine di Precedenza nelle Metriche

RISPOSTA CORRETTA: Pianificando, prima decido fino a che livello voglio soddisfare le attese, e poi calcolo il costo corrispondente, iterando fino a quanto i costi preventivati siano entro gli eventuali vincoli contrattuali.

ERRATO:

- Prima decidere quanto spendere, poi vedere dove andare
 - Nessun ordine di precedenza (metriche indipendenti)
-

5. Milestone e Baseline

5.1 Milestone

DEFINIZIONE: Punto di arrivo strategico, pianificato e misurabile del software, corrispondente al raggiungimento di un insieme di obiettivi definito.

Caratteristiche:

- Elemento di pianificazione
- Stato d'avanzamento fondamentale

- Punto obiettivo nel tempo
- Concretizzata da almeno una baseline

5.2 Baseline

DEFINIZIONE: Stato di riferimento dello sviluppo del prodotto in una data fase di progetto. Base verificata, approvata e garantita di un insieme di CI.

Caratteristiche:

- Istanza una milestone
- Versione da cui non si può retrocedere
- Base per successivo incremento
- Collegata a controllo configurazione

5.3 Technology Baseline (TB) vs Product Baseline (PB)

Technology Baseline (RTB)

Obiettivi:

- Fissare requisiti che fornitore si impegna a soddisfare
- Motivare tecnologie, framework, librerie selezionate
- Dimostrare adeguatezza e fattibilità tramite PoC

Artefatti:

- Analisi dei Requisiti (documento principale)
- Proof of Concept (demo eseguibile)

CROCETTA IMPORTANTE: La TB, essendo associata a un PoC, corrisponde bene alla milestone "demonstrable"; la PB invece sta tra "usable" e "ready" ✓

Product Baseline (PB)

Obiettivi:

- Vagliare maturità baseline architetturale
- Design definitivo coerente con PoC
- Avanzamento sostanziale di codifica (prodotto dimostrabile)

Artefatti:

- Specifica Tecnica (documento principale)
 - MVP (Minimum Viable Product)
-

6. Diagrammi UML

6.1 Diagrammi delle Classi - Relazioni

Associazione

- Linea continua tra classi

- Include molteplicità (1, 0..1, 0..*, *)
- Può essere unidirezionale (freccia)

Aggregazione

- Diamante **vuoto** lato aggregante
- Relazione "part-of" debole
- Parti possono essere condivise
- Parti possono esistere senza tutto

Composizione

- Diamante **pieno** lato compositore
- Relazione "part-of" forte
- Parti NON condivise
- Parti NON esistono senza tutto

CROCETTA IMPORTANTE: La composizione è un tipo di dipendenza dove una classe è responsabile dell'intero ciclo di vita degli oggetti dell'altra ✓

Dipendenza

- Linea **tratteggiata** con freccia
- Una classe usa temporaneamente l'altra
- Modifica fornitore può influenzare client

Ereditarietà

- Freccia **vuota** verso superclasse
- Relazione "is-a"
- Genera accoppiamento **stretto** (NON lasco)

6.2 Validità Sintattica UML

Controlli importanti:

- Observer: Subject deve avere attach/detach/notify
- Singleton: costruttore privato, getInstance() statico
- Adapter: relazioni corrette tra Client, Target, Adaptee
- Relazioni: frecce, diamanti, linee nel verso giusto

6.3 Diagrammi di Attività

Elementi fondamentali:

- **Nodo iniziale:** cerchio nero pieno
- **Nodo finale:** cerchio nero con bordo
- **Attività:** rettangolo arrotondato
- **Decision:** diamante con condizioni
- **Fork/Join:** barra nera per parallelismo
- **Flusso:** frecce tra elementi

6.4 Diagrammi dei Casi d'Uso

Elementi:

- **Attore:** stick figure (ruolo utente)
- **Caso d'uso:** ovale (funzionalità)
- **Sistema:** rettangolo contenitore

Relazioni:

- **Include:** <<include>> sempre eseguito
 - **Extend:** <<extend>> opzionalmente eseguito
 - **Generalizzazione:** specializzazione attori/casi d'uso
-

7. Concetti Teorici Fondamentali

7.1 Proof-of-Concept (PoC)

DEFINIZIONE CORRETTA: Il PoC è un eseguibile, sviluppato a inizio progetto, con tempi e costi strettamente limitati, per comprendere al meglio i requisiti tecnici e la fattibilità delle soluzioni tecnologiche individuate.

NON È:

- Baseline avanzata del prodotto finale
- Prototipo usa-e-getta sviluppato in velocità
- Demo per il cliente

7.2 Copertura dei Requisiti

DEFINIZIONE: Percentuale di requisiti software associati a specifici artefatti/componenti/parti del prodotto come determinati dalla progettazione; test confermano effettivo grado di copertura.

Caratteristiche:

- Determinabile durante progettazione
- Non solo superamento test funzionali
- Include tracciamento requisiti → componenti → test

7.3 Architettura Software

DEFINIZIONE: Decomposizione del sistema in componenti, loro organizzazione, interfacce per interazione e regole di composizione.

Relazioni:

- **Design Pattern:** soluzione teorica a problema ricorrente
 - **Framework:** componenti concrete prefabbricate
 - Framework spesso implementano design pattern
-

8. Processi di Ciclo di Vita

8.1 Standard ISO/IEC 12207

Definizione Processo: Insieme di attività correlate e coese che trasformano ingressi in uscite secondo regole fissate, consumando risorse.

8.2 Categorie Processi

Processi Primari

- **Acquisizione:** attività committente
- **Fornitura:** attività fornitore
- **Sviluppo:** analisi, design, codifica, test
- **Operazione:** utilizzo software
- **Manutenzione:** modifiche post-delivery

Processi di Supporto

- **Documentazione:** produzione documenti
- **Gestione Configurazione:** controllo modifiche
- **Garanzia Qualità:** assicurazione qualità
- **Verifica:** conferma soddisfacimento requisiti
- **Validazione:** conferma utilizzo previsto

Processi Organizzativi

- **Gestione:** coordinamento e controllo
 - **Infrastruttura:** ambiente di supporto
 - **Miglioramento:** enhancement continuo
-

9. Test e Verifica

9.1 Test di Unità

DEFINIZIONE CORRETTA: Una "unità" è nozione architetturale: parte del tutto, piccola abbastanza da essere verificata individualmente (senza dipendenze significative), ma grande abbastanza da meritare verifica (correttezza non accertabile "a vista").

Caratteristiche:

- NON dipende dal linguaggio programmazione
- È concetto architetturale ben definito
- Focus su parti verificabili individualmente

9.2 Verifica vs Validazione

Verifica

Obiettivo: Confermare che prodotto soddisfa requisiti specificati **Caratteristiche:**

- Controllo sistematico, disciplinato, misurabile
- Interesse interno azienda
- Gestita da verificatori progetto

Validazione

Obiettivo: Confermare che prodotto soddisfa utilizzo previsto **Caratteristiche:**

- Coinvolge cliente finale
- Test funzionali e collaudo
- Accerta conformità prodotto rispetto requisiti

9.3 Tecniche di Verifica

Walkthrough

- Analisi statica a largo spettro
- Attività collaborativa onerosa ma efficace
- Codice esaminato esaustivamente
- Attività pianificata e documentata

Inspection

- Analisi mirata e strutturata
 - Localizza errori segnalati in lista controllo
 - Meno onerosa ma richiede esperienza
 - Lista controllo progressivamente estesa
-

10. Metodi Agili

10.1 Agile vs Incrementale

RELAZIONE CORRETTA: Entrambi guardano nella stessa direzione, ma sviluppo agile contempla rischio di iterazioni distruttive, che invece sviluppo incrementale cerca in ogni modo di evitare.

Sviluppo Incrementale:

- Consegne successive aggiungono funzionalità
- Evita modifiche distruttive
- Baseline stabile sempre disponibile

Sviluppo Agile:

- Accetta modifiche/refactoring significativi
- Iterazioni possono essere distruttive
- Flessibilità su stabilità

ERRATO:

- Sono incompatibili
- Sono identici

- Agile è automaticamente incrementale
-

11. Architetture Software

11.1 Monolite vs Microservizi

Quando Microservizi sono vantaggiosi

CORRETTO: Secondo Legge di Conway, organizzazioni con gruppi di sviluppo indipendenti beneficiano di architettura a microservizi.

Caratteristiche Microservizi:

- Servizi piccoli e indipendenti
- Team indipendenti per servizio
- Deployment indipendente
- Database per servizio

IMPORTANTE: Microservizi NON sempre preferibili, monolite NON obsoleto

Domande Tipo e Risposte ESATTE

SOLID Principles

1. **Single Responsibility Principle** è noto come: coesione ✓
2. **Liskov Substitution Principle** è associabile a: Design by Contract ✓
3. **Open-Close con variabili globali**: possibile nei termini del linguaggio ✓

Coesione/Accoppiamento

1. **Massimo grado coesione sistema**: dipende dalla struttura (FALSO per sistemi complessi)
2. **Accoppiamento è**: caratteristica negativa da ridurre ✓
3. **Composizione**: responsabile ciclo vita oggetti ✓

Metriche

1. **Efficacia**: grado conformità con attese ✓
2. **Efficienza**: funzione inversa risorse impiegate ✓
3. **Ordine precedenza**: prima attese, poi costi ✓

Baseline/Milestone

1. **TB vs PB**: TB demonstrable, PB usable/ready ✓
2. **Milestone**: punto arrivo strategico ✓
3. **Baseline**: stato riferimento verificato ✓

Pattern

1. **Template Method con composizione**: FALSO (usa ereditarietà) ✓

2. **Observer**: Subject notifica Observer ✓
3. **Validità UML**: controllare sintassi specifica pattern

Test

1. **Test unità**: concetto architetturale ben definito ✓
2. **Agile vs Incrementale**: stessa direzione, agile accetta distruzione ✓